

Two-Stage Fusion for Routing and Classification in Parameter-Efficient Continual Learning

Nguyễn Thiên Trường and Tống Quang Thái

University of Engineering and Technology,
Vietnam National University, Hanoi, Vietnam
24022480@vnu.edu.vn, 24022450@vnu.edu.vn

Abstract. Parameter-efficient continual learners maintain task-specific adapters whose selection affects both routing and classification. We augment HRM-PET with a random-projection (RP) head fitted by ridge regression over fixed first-task LoRA features. Its sufficient statistics are accumulated task by task without retaining examples. The same RP scores support two inference-time operations: task-level fusion with the initial adapted prediction before re-matching, and class-level fusion with the final logits using a top-two-margin gate. Across three benchmarks and four seeds, DRM-proposal accuracy increases by 1.10–2.26 points and classification accuracy by 0.69–1.50, with no consistent improvement in forgetting or backward transfer. Single-seed ablations across five backbones show that the routing contribution depends on the gate. Running the two stages apart over four seeds separates them: routing alone is positive on ImageNet-R and CUB-200 and unresolved on CIFAR-100, while the gate carries most of the gain on the first two datasets and costs accuracy on the third. A grid over the two mixing weights shows that classification accuracy is nearly flat in the routing weight, so the reported value is not a resolved optimum.

Keywords: Continual learning · Parameter-efficient tuning · Parameter routing · Random projection · Signal fusion.

1 Introduction

Continual learning asks a model to absorb a stream of tasks without losing what earlier ones taught it. The dominant recent answer freezes a large pre-trained backbone and attaches a small set of task-specific parameters — prompts, adapters, low-rank updates — so that a new task writes into fresh capacity instead of overwriting old weights. Old parameters survive and no exemplars need be stored, which is why parameter-efficient tuning now leads the exemplar-free setting [4, 5, 3]. Keeping the old parameters is not the same as keeping the old accuracy, which is what the metrics measure.

The design moves the difficulty rather than removing it. At inference the task identity is unknown, so before the model can classify it must choose which parameter set to attach. That choice is itself a prediction, and every later computation inherits it: attach the wrong adapter and the classifier is asked to work

from features it was never adapted for. Successive methods have improved the choice — larger pools, attention-weighted mixtures, explicit re-matching stages that revisit an initial guess — but the choice still rests on evidence drawn from one place, the backbone’s own predictions.

We ask what a second, differently derived source of evidence would be worth, and find one available for the price of a forward pass: the features such a learner computes anyway, pushed through a frozen random projection and read by a ridge classifier fitted in closed form from accumulated second-order statistics, in the manner of RanPAC [2]. That head stores no examples, changes no training procedure, and — the property that matters here — makes different mistakes from the routing module. We reuse it at the two points where the pipeline commits: to revise the task proposal before re-matching, and to blend with the final class logits under a per-sample weight that grows as the primary classifier’s top-two margin narrows. Taking HRM-PET [1] as the pipeline to build on, we measure what each operation contributes, and say so where the evidence does not settle the question.

Contributions. We examine unused complementary evidence in HRM-PET [1] and show that a better task proposal does not automatically become a better class prediction, because the shared head frequently predicts the correct class in spite of a wrong route (Sect. 2.1). We show that a second-order head over a frozen random projection of the existing features, fitted in closed form and never by gradient descent, supplies evidence with partially different errors, and we fuse the two at the routing and the classification level alike, entirely at inference time. We evaluate on three datasets over four seeds and report by axis: routing improves by +1.10 to +2.26 points and classification by +0.69 to +1.50, each several times the seed-to-seed standard deviation, while the two retention metrics show no consistent improvement (Sect. 4). We reconstruct the full grid the original work covers — four datasets by five pre-training configurations, identified from traces in its released code — where all 20 cells improve on both accuracy axes, over baselines spanning 31.8 to 93.8 Acc@1 (Sect. 4.4).

2 Related Work

Parameter pools and the selection they require. L2P [4] and Dual-Prompt [5] set the shape most parameter-efficient continual learners still follow: freeze the backbone, keep a pool of lightweight parameter sets, and pick from it at inference using a query from the bare backbone. CODA-Prompt [7] replaces the discrete pick with an attention-weighted combination; HiDe-Prompt [3] separates within-task prediction, task-identity inference and task-adaptive prediction and optimises them apart, observing that identity inference, not within-task classification, is what limits the pipeline. HRM-PET [1], our baseline, substitutes LoRA adapters [6] for prompts and adds re-matching stages that revisit the initial decision. Later work enlarges the expert family: MoTE [14] combines task-

specific experts, and DLEPEM [15] combines general and adapted prototypes for matching.

Closed-form solutions in continual learning. A separate line replaces gradient descent on the classifier with a solution written in closed form, which removes the mechanism by which sequential updates overwrite one another. Nearest-class-mean prototypes are the simplest instance, and APER [8] adapts the backbone on the first task only, then reads frozen features. Analytic continual learning states the problem as least squares directly: ACIL [20] recasts class-incremental learning as a recursive least-squares problem in which a correlation matrix rather than a sample buffer carries the past, yielding a model equal to the one a joint fit would produce, and DS-AL [21] adds a second stream to recover nonlinear capacity that a single linear solve gives up. RanPAC [2] reaches the same form from the feature side: frozen features through a fixed random projection, second-order statistics accumulated across tasks, one ridge system solved.

Our head belongs to this family and we claim no novelty in it: the projection, the Gram accumulation and the ridge solve are RanPAC’s. What differs is its role. These methods use the closed-form classifier as the whole predictor and discard routing; we keep the routing pipeline and consult the solved head twice.

Correcting the classifier after the fact. Class-incremental classifiers lean towards recent classes, and the survey of Masana et al. [9] locates the bias in both the bias terms and the classifier weight norms. Corrections follow: iCaRL [10] classifies to the nearest exemplar mean; BiC [11] learns a per-task affine map on the logits from a held-out split; Weight Aligning [12] rescales new-class weight norms to match the old; LUCIR [13] normalises features and weights before the softmax. We keep the adapters, auxiliary classifier and shared head fixed and retain no exemplars, though current-task data are still needed to fit the RP head, whose scores we fuse at inference at a cost included in the reported total.

2.1 HRM-PET: The Baseline We Build On

We use the released HRM-PET [1] inference path. Diagnostics quoted here are seed-42 measurements, separate from the evaluation of Sect. 4.

Task-identity inference (TII). The input image is passed through the *bare* backbone, with no adapter attached, to obtain a query feature $q(x) = \phi(x; \theta_{\text{ptm}})$, writing ϕ for the backbone under whatever adapter it is given. That feature enters an auxiliary classifier trained with cross-entropy, which gives unnormalised class logits $u = g(q(x); \theta_\omega)$. The initial match is then $t_0 = \mathcal{T}(\arg \max_c u_c)$.

Re-matching (DRM and CRM). The first adapted pass attaches p_{t_0} and produces $r = g_{\theta_g}(\phi(x; p_{t_0}, \theta_{\text{ptm}}))$. DRM proposes the task of $\arg \max r$ and evaluates its adapter when it differs from t_0 ; our fusion modifies this proposal, not

the preceding bare TII pass. CRM selects low-confidence samples from r by the baseline’s generalised-entropy threshold and compares the current logits, under the baseline log-sum-exp score, with the adapter of the second-ranked class of r ; it is confidence-triggered, not restricted to cases where the two task identities agree. Both reuse trained modules, can require further adapter forwards, and are heuristics: neither guarantees that accuracy cannot decrease.

Shared classification head. The selected parameter set is attached to the backbone, and the shared head produces a score vector over all classes seen so far, $\ell = g(\phi(x; p_{\hat{t}}, \theta_{\text{ptm}}); \theta_g) \in \mathbb{R}^{|\mathcal{Y}|}$, predicting $\hat{y} = \arg \max_c \ell_c$, where $\hat{t}$ is the identity after DRM and CRM. Classes not yet seen are masked with $-\infty$.

Two weaknesses this paper acts on. The baseline draws its routing evidence from TII and the shared head alone. To see what a second source would be worth we built the head of Sect. 3.2 on HRM-PET’s own LoRA features and used it as a router: at seed 42 on ImageNet-R, TII routes 63.80% of samples correctly, the DRM proposal 77.79% and the RP head 77.12%, while the union of TII and the head reaches 82.99%. Since TII is wrong on 36.20% of the set, the head repairs 53.0% of its errors, against 62.0% and 46.9% on CIFAR-100 and CUB-200. The union is an oracle that no router we build attains, and it compares bare TII with RP rather than the actual Stage 1 inputs.

A better route need not become a better prediction: ℓ comes from the shared head, which spans every class seen so far, so a mis-routed sample is often classified correctly anyway. The RP head is moreover a complete classifier that a routing-only variant would discard: on ImageNet-R the routed head reaches 74.31% and the projection head 70.08%, yet their union is 78.89%, the head alone contributing 4.58 points of the test set and 2.43 and 3.87 on the other two. These measurements give the two components described next.

3 Method

3.1 Problem Statement

Class-incremental learning (CIL) learns a sequence of T tasks without forgetting earlier ones. The t -th task consists of n_t samples $\mathcal{D}_t = \{(x_i^t, y_i^t)\}_{i=1}^{n_t}$ drawn from a class set $\mathcal{C}_t$, where $x_i^t \in \mathcal{X}$ is an input image and $y_i^t \in \mathcal{Y}$ its label.

Three constraints define the setting. At step t only $\mathcal{D}_t$ is available for training. The class sets are disjoint, $\mathcal{C}_t \cap \mathcal{C}_{t'} = \emptyset$ for all $t \neq t'$, and at inference the *task identity is unknown*: the model must discriminate among all classes seen so far through a single shared classification head. Finally, this work operates in the exemplar-free setting: images from past tasks are never retained.

In methods based on parameter-efficient tuning (PET) [4, 5, 3], a pre-trained backbone h_{ptm} with parameters θ_{ptm} is frozen and a lightweight parameter pool $\mathcal{P} = \{p_1, \dots, p_T\}$ is maintained, each set trained on one task; here every p_t is a LoRA adapter [6]. The shared classification head is denoted g_{θ_g} . Because the task

identity is unknown at inference, the system is forced to *choose* which parameter set to attach to the backbone before classifying, and everything downstream depends on that choice.

We distinguish four class-score vectors: bare TII scores u , initial adapted shared-head logits r , final re-matched logits ℓ , and RP scores s . The class-to-task map is $\mathcal{T}(\cdot)$; t_0 is the initial TII match and $\hat{t}_{\text{DRM}}$ the task proposed for direct re-matching. We write $z(\cdot)$ for per-sample standardisation with the population standard deviation floored at 10^{-6} .

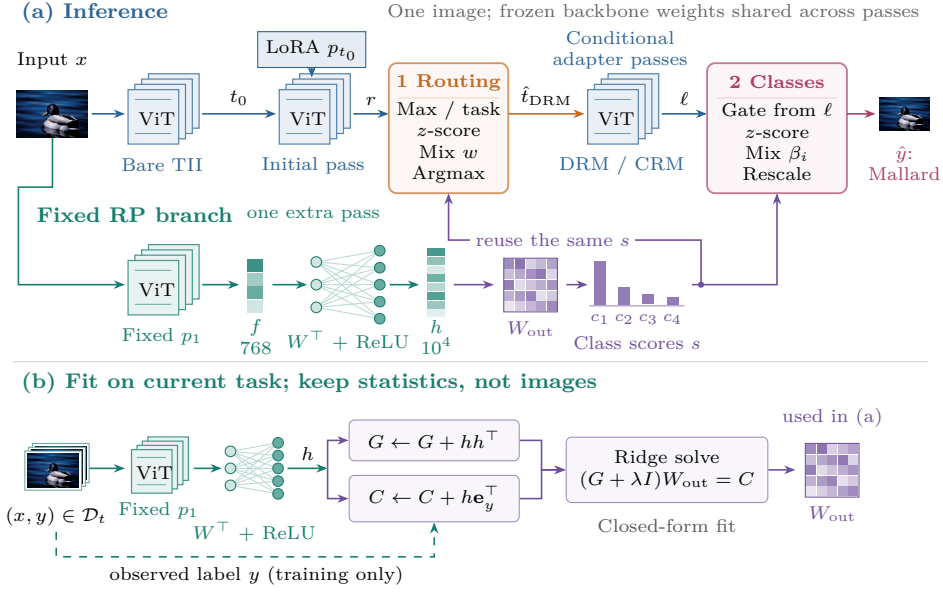


Fig. 1. Two-stage fusion. All inference passes use the same image; adapted passes share head g . TII selects t_0 ; the projection W is frozen. Stage 1 standardises task-wise maxima of r and s ; Stage 2 uses the top-two probability margin of ℓ for its continuous gate (Eqs. 6–8), then predicts by argmax. DRM runs if $\hat{t}_{\text{DRM}} \neq t_0$; CRM uses low-confidence r and its second-ranked class’s task, comparing logits by log-sum-exp (not at task 1). Only seen classes/tasks participate; score bars are schematic. Photo: T. G. Barnes/USFWS (public domain).

3.2 The Second Source of Evidence

We fit an RP head after each task and reuse its scores in two inference-time fusion operations. HRM-PET parameters are not retrained. Figure 1 distinguishes the initial adapted logits from the bare TII scores and shows both reuse paths.

The RP head follows RanPAC [2]. Features $f \in \mathbb{R}^d$, $d = 768$, are column vectors from ViT-B/16 with the first task’s LoRA adapter (p_1 here, index 0

in code), fixed during accumulation and inference and unnormalised; changing adapters between tasks would mix incompatible spaces.

Projection and second-order statistics. The feature is projected into a high-dimensional space of size $M = 10^4$ and passed through a nonlinearity, and the projected features are accumulated into two matrices:

$$h = \text{ReLU}(W^\top f) \in \mathbb{R}^M, \quad W \in \mathbb{R}^{d \times M}, \quad W_{jk} \sim \mathcal{N}(0, 1), \quad (1)$$

$$G = \sum_n h_n h_n^\top \in \mathbb{R}^{M \times M}, \quad C = \sum_n h_n \mathbf{e}_{y_n}^\top \in \mathbb{R}^{M \times |\mathcal{Y}|}, \quad (2)$$

where $\mathbf{e}_y$ is the basis vector of class y . Each current-task training sample contributes once; G and C are accumulated in float64. W is generated once with seed 1993 and then frozen.

The nonlinear expansion permits richer decision functions than a linear head on f . Without ReLU, $W^\top f$ remains linear in f and adds no separating power. This does not guarantee that the expanded data become linearly separable.

Classification by ridge regression. The output weights are the closed-form solution of a regularised least-squares problem:

$$W_{\text{out}} = (G + \lambda I)^{-1} C \in \mathbb{R}^{M \times |\mathcal{Y}|}, \quad s = W_{\text{out}}^\top h \in \mathbb{R}^{|\mathcal{Y}|}. \quad (3)$$

The implementation solves the regularised system rather than inverting it.

Two inherited properties. Both G and C in (2) are *sums* over samples, so for a *fixed* feature map they, and with them W_{out} , do not depend on the order in which samples arrived: nothing an earlier task contributed is overwritten, and the head after T tasks equals the ridge solution of a joint fit over every feature seen. Their sizes do not grow with the sample count and no per-sample feature is retained, so the exemplar-free protocol follows from the structure of the computation. This is weaker than an absence of forgetting and we claim no more: adding a task adds classes competing for the same arg max, so accuracy on old classes can fall while their statistics sit untouched, and the feature map is not fixed across orderings, since features come from the first task’s adapter. The property rules out one mechanism of forgetting, not the phenomenon.

3.3 Two-Stage Fusion

Both stages blend vectors on different scales: the ridge scores s span roughly one unit and the logits ℓ roughly thirty-five, so a direct blend would let ℓ dominate at every weight. Each source is standardised per sample:

$$z(v) = \frac{v - \text{mean}(v)}{\sigma_\epsilon(v)}, \quad \sigma_\epsilon(v) = \max\{\text{std}(v), 10^{-6}\}, \quad (4)$$

where std is the population standard deviation. Moments are over seen tasks in Stage 1 and seen classes in Stage 2, excluding unseen entries.

First stage: fusion at the routing level. Stage 1 runs after the initial adapted pass and before DRM. Its input is r , the shared-head logits under the adapter selected by bare TII, not auxiliary logits u . Both r and RP scores s are mapped to seen tasks by $r_t^{\text{task}} = \max_{c \in \mathcal{C}_t} r_c$ and $s_t^{\text{task}} = \max_{c \in \mathcal{C}_t} s_c$ for $t = 1, \dots, \hat{T}$, with $\hat{T}$ the number of tasks seen so far; classes of unseen tasks never enter either maximum. The two vectors are blended after standardisation:

$$\text{fused} = w z(r^{\text{task}}) + (1 - w) z(s^{\text{task}}), \quad \hat{t}_{\text{DRM}} = \arg \max_t \text{fused}_t, \quad w = 0.7. \quad (5)$$

For one seen task the standard deviation is zero, so the floor keeps the operation finite. The fused task replaces the baseline DRM proposal $\mathcal{T}(\arg \max r)$; the initial TII match, CRM candidate rule, confidence test and shared head are unchanged. At $w = 1$ the ordering is that of r^{task} , recovering that proposal; the full baseline also needs $\beta = 0$.

Second stage: gated fusion at the classification level. Because ℓ comes from the shared head, it frequently predicts the correct class in spite of a wrong routing decision, so we use the projection head itself as a secondary classifier, this time taking its class scores s . A fixed blend would alter confident primary predictions; we instead reduce the RP share as the top-two probability margin grows, while recognising that confident predictions can still be wrong:

$$\beta_i = \beta \left(1 - \frac{p_{(1)} - p_{(2)}}{p_{(1)} + p_{(2)}} \right), \quad \beta = 0.5, \quad (6)$$

where $p_{(1)} \geq p_{(2)}$ are the two largest probabilities of $\text{softmax}(\ell)$ over the classes seen so far. Dividing by the sum makes the quantity depend only on the ratio $p_{(2)}/p_{(1)}$, so the gate measures how threatened the top class is rather than how much mass the pair holds. The RP share falls to zero as the top class dominates and reaches β when the two are equal: a continuous weight, not a binary test.

The blend reuses (4) and then maps the result back onto the scale of ℓ :

$$m_i = (1 - \beta_i) z(\ell_i) + \beta_i z(s_i), \quad (7)$$

$$\hat{\ell}_i = m_i \cdot \sigma_\epsilon(\ell_i) + \text{mean}(\ell_i) \in \mathbb{R}^{|\mathcal{Y}|}. \quad (8)$$

The positive affine map-back preserves rankings and restores a reference scale for cross-entropy, not a calibration guarantee: for standardised nondegenerate vectors with correlation ρ and the floor inactive, the mixture variance is $(1 - \beta_i)^2 + \beta_i^2 + 2\beta_i(1 - \beta_i)\rho$, which need not equal one. At $\beta = 0$ this stage leaves ℓ unchanged.

Cost. At inference the head costs one extra adapter forward per sample, serving both stages. That is the head’s own cost, not the pipeline’s difference, which we did not measure: changing the DRM proposal can change the number of adapter evaluations.

Per task the head accumulates G and C over the current task’s training split and solves one ridge system, so its footprint is set by M , not by how many samples have been seen — and it is not small. At $M = 10^4$ the Gram matrix holds 10^8 entries, 800 MB in the float64 the solve requires, and the Cholesky factor with the regularised copy brings the peak near 2.4 GB. A complete ten-task evaluation on ImageNet-R takes 283 s on one RTX 4090 at batch 24, a total we did not instrument by phase.

4 Experiments

4.1 Setup

The backbone is a ViT-B/16 pre-trained on ImageNet-21K and adapted with LoRA of rank 8. The four-seed study uses Split-ImageNet-R, Split-CIFAR-100 and Split-CUB-200 in 10 tasks each; the grid of Sect. 4.4 swaps CUB-200 for Split-ImageNet-A and 5-Datasets, a 5-task sequence. The five pre-training settings are Sup-21K, MoCo-v3, iBOT-1K, iBOT-21K and DINO. Seeds are 42 to 45, with the LoRA pool and the TII module retrained from scratch for each; baseline and proposed method are then evaluated from the same checkpoints through the same code path. The projection head uses $M = 10^4$, a ReLU non-linearity and $\lambda = 10^4$, with no temperature scaling; fusion starts at task 1; the baseline CRM settings are preserved (**En=gen**, threshold -10 , two class-derived candidate tasks).

Acc@task in the released evaluator measures the DRM task proposal. In the default class-based routing path, CRM can change the output logits without updating the task identity passed to this metric; it is therefore not a measure of final adapter selection after CRM. Acc@1, Acc@5 and Loss use the final classification logits. Writing $a_{i,t}$ for classification accuracy on task i at stage t , the retention measures are

$$\text{Forgetting} = \frac{1}{T-1} \sum_{i=1}^{T-1} \left(\max_{i \leq t \leq T} a_{i,t} - a_{i,T} \right), \quad (9)$$

$$\text{Backward} = \frac{1}{T-1} \sum_{i=1}^{T-1} (a_{i,T} - a_{i,i}). \quad (10)$$

Both averages exclude the last task, and the maximum includes only stages at which task i has been learned. Lower is better for Loss and Forgetting.

Sanity check. $w = 1.0$ in (5) restores the original DRM proposal and $\beta = 0$ in (7) leaves $\hat{\ell} = \ell$; with both set the method reproduces the HRM-PET baseline digit for digit on all three datasets — 77.7914, 74.0191, 1.2305, the rest agreeing to the fourth decimal — so every difference in Table 1 comes from the added components, not the evaluation path.

4.2 Main Results

Table 1 reports all three datasets over four seeds. The change column is pairwise — taken per seed, then averaged — so its standard deviation describes the stability of the improvement, not the seeds’ difficulty.

Table 1. Results on the three Split datasets, mean $\pm$ standard deviation over seeds 42–45.

Dataset	Metric	HRM-PET	Proposed	Change
<i>ImageNet-R</i>	Acc@task	77.58 $\pm$ 0.33	79.85 $\pm$ 0.18	+2.264 $\pm$ 0.158
	Acc@1	73.94 $\pm$ 0.48	75.18 $\pm$ 0.24	+1.236 $\pm$ 0.416
	Acc@5	86.71 $\pm$ 0.21	88.02 $\pm$ 0.17	+1.312 $\pm$ 0.148
	Loss	1.23 $\pm$ 0.01	1.19 $\pm$ 0.01	−0.040 $\pm$ 0.002
	Forgetting	3.36 $\pm$ 0.56	3.30 $\pm$ 0.35	−0.056 $\pm$ 0.334
<i>CIFAR-100</i>	Backward	−3.16 $\pm$ 0.61	−3.20 $\pm$ 0.42	−0.047 $\pm$ 0.282
	Acc@task	89.77 $\pm$ 0.11	90.87 $\pm$ 0.09	+1.103 $\pm$ 0.066
	Acc@1	89.78 $\pm$ 0.03	90.47 $\pm$ 0.05	+0.690 $\pm$ 0.063
	Acc@5	98.65 $\pm$ 0.03	98.86 $\pm$ 0.07	+0.217 $\pm$ 0.057
	Loss	0.39 $\pm$ 0.00	0.39 $\pm$ 0.00	−0.000 $\pm$ 0.001
<i>CUB-200</i>	Forgetting	3.89 $\pm$ 0.08	3.74 $\pm$ 0.12	−0.156 $\pm$ 0.098
	Backward	−3.87 $\pm$ 0.07	−3.73 $\pm$ 0.11	+0.142 $\pm$ 0.097
	Acc@task	93.12 $\pm$ 0.14	94.34 $\pm$ 0.10	+1.226 $\pm$ 0.149
	Acc@1	86.38 $\pm$ 0.25	87.88 $\pm$ 0.27	+1.495 $\pm$ 0.353
	Acc@5	97.04 $\pm$ 0.10	97.44 $\pm$ 0.13	+0.400 $\pm$ 0.103
	Loss	0.57 $\pm$ 0.00	0.53 $\pm$ 0.01	−0.036 $\pm$ 0.004
	Forgetting	2.54 $\pm$ 0.35	2.18 $\pm$ 0.12	−0.359 $\pm$ 0.325
	Backward	−2.42 $\pm$ 0.39	−2.14 $\pm$ 0.13	+0.279 $\pm$ 0.391

Intervals are paired 95% Student- t , $\bar{\Delta} \pm 3.182 s_{\Delta}/\sqrt{4}$: imprecise at four seeds, and assuming approximately normal seed-wise differences. Correlated metrics mean we discuss axes rather than count confirmations.

Routing (Acc@task): improved. The gain is +1.10 to +2.26 points, with positive paired intervals on all three datasets. This is the axis the first stage acts on directly.

Classification (Acc@1, Acc@5): improved. Acc@1 rises by +0.69 to +1.50 points and Acc@5 by +0.22 to +1.31, with positive paired intervals for both metrics on all three datasets. The full method’s Acc@1 improvement repeats across every seed. These aggregate gains do not establish how many routing repairs also repair classification.

Retention (Forgetting, Backward): small, and mostly unresolved. Read as paired differences over the four seeds, the 95% interval from t_3 covers zero in

all six retention cells; the narrowest, Forgetting on CIFAR-100, is -0.156 with interval $[-0.312, +0.001]$, which reaches zero only just. The two metrics are near mirror images — -0.156 against $+0.142$ on CIFAR-100, -0.359 against $+0.279$ on CUB-200 — and are not independent replications. We claim no consistent improvement here, and no equivalence either: four seeds give intervals wide enough to hold differences that would matter, and we fixed no margin in advance. The single-seed decomposition in Sect. 4.4 is descriptive: final accuracy on earlier tasks rises in every grid cell, while Backward changes little because accuracy at learning time also rises.

4.3 Placement Against Prior Methods

Table 2 places the method among the baselines the original work reports. Its first eight rows are quoted from Table 1 of [1] under Sup-21K, averages over three seeds; the last two are our own runs at seed 42 through the code path of Sect. 4.1.

Table 2. Final average accuracy (%) under Sup-21K. Rows above the rule are quoted from Table 1 of [1], averaged over three seeds; the two below are ours at seed 42, from a different implementation and seed set.

Method	CIFAR-100	ImageNet-R	ImageNet-A	5-Datasets
DualPrompt [5]	86.60	68.79	39.76	76.09
S-Prompt++ [16]	88.81	69.68	39.10	83.38
CODA-Prompt [7]	86.94	70.03	30.96	62.54
LAE [17]	85.10	72.25	40.36	64.16
CPrompt [18]	83.18	69.24	37.60	71.72
InfLoRA [19]	85.28	72.24	42.12	71.97
HiDe-LoRA [3]	88.43	71.90	42.18	93.25
HRM-PET [1]	89.45	73.86	44.28	93.99
HRM-PET, our run	89.77	74.02	45.85	91.37
Proposed	90.53	75.51	47.42	93.77

Our reproduction lands within 0.32 points of the published figure on CIFAR-100 and 0.16 on ImageNet-R, and over four seeds gives 73.94 ± 0.48 there against the published 73.86 ± 0.14 ; that agreement is what makes quoting the other rows meaningful. It is 1.57 above on ImageNet-A and 2.62 below on 5-Datasets, so those columns carry a reproduction gap the quoted rows do not share. Quoted rows come from another implementation and seed set: they place the work, they are not a controlled comparison, which remains the paired difference in Table 1.

4.4 The Full Grid

The original work covers four datasets by five pre-training configurations; we rebuilt all twenty cells, evaluating both methods through one code path at seed

42. Every cell improves on both accuracy axes: Acc@task by +0.97 to +4.57 and Acc@1 by +0.59 to +3.58, against baselines spanning 31.8 to 93.8 Acc@1. Retention is descriptive rather than conclusive: final accuracy on earlier tasks rises in all twenty cells, by +1.83 on average, while accuracy at learning time rises by +1.58, so Backward, their difference, stays small. It worsens in five cells, which by that identity means learning-time accuracy rose more there — a widening gap, not a fall in final accuracy.

4.5 Ablation

We ablate *cumulatively*, as the original work does, on ImageNet-R at seed 42 across its five pre-training configurations, whose baselines run from 68.47 to 74.02 Acc@1. Only differences are reported, since only they carry the argument.

Routing gains depend on the gate. Each stage alone beats the baseline on every backbone — routing by +0.29 to +0.74, the classifier by +0.51 to +3.07. Adding routing to class fusion moves Acc@1 by -0.07 to $+0.02$ with the gate off and by $+0.13$ to $+0.77$ with it on, positive on all five backbones. The classifier’s own contribution is smallest by far on Sup-21K, $+0.51$ against $+2.23$ to $+3.07$ elsewhere — the backbone every design decision was taken on.

The two stages, run apart over four seeds. On all three datasets we ran ungated class-only fusion ($w = 1$, $\beta = 0.5$, no gate), the same with the margin gate, and the full method, at seeds 42–45. Only w separates the middle condition from the full method, so that pair isolates routing and the other isolates the gate.

Routing is $+0.083$ [$+0.026$, $+0.140$] on ImageNet-R, -0.030 [-0.085 , $+0.025$] on CIFAR-100 and $+0.156$ [$+0.055$, $+0.257$] on CUB-200. The gate is $+0.603$ and $+0.355$ on the first two, both excluding zero, and -0.284 [-0.570 , $+0.002$] on CUB-200. Each pair sums to the full-against-class-only change, $+0.69$, $+0.33$ and -0.13 , to within 0.005. Routing is positive on two datasets and unresolved on the third, and the loss on CUB-200 is the gate’s: routing there is the largest of the three. The first stage remains the only component that moves Acc@task.

4.6 Sensitivity to the Mixing Weights

The two weights were fixed sequentially, w before the second stage existed and β afterwards, so the second stage may have shifted the optimum of the first. Table 3 reports Acc@1 over the joint grid on ImageNet-R at seed 42 with the margin gate active.

On this grid $\beta = 0.5$ is best at every w and the nine cells span 0.49 points, so choosing the two in sequence would have cost nothing measurable here. Acc@1 is nearly flat in w while the routing axis is not: at $\beta = 0.5$ the three values of w span 0.11 points of Acc@1 but 1.01 of Acc@task, 80.39, 79.97 and 79.37. That one-to-nine ratio is between two aggregate gains, so it says only that most of

Table 3. Acc@1 over the $w \times \beta$ grid, ImageNet-R at seed 42, full method with the margin gate.

	$\beta = 0.3$	$\beta = 0.5$	$\beta = 0.7$
$w = 0.6$	75.28	75.61	75.29
$w = 0.7$ (reported)	75.22	75.51	75.24
$w = 0.8$	75.12	75.50	75.27

the routing accuracy bought by lowering w does not reach Acc@1, not that one routing repair in nine repairs a class.

The grid’s maximum is $w = 0.6$ at 75.61, not the reported $w = 0.7$ at 75.51, so we ran both over seeds 42–45 at fixed β and gate. On Acc@task $w = 0.6$ leads by +0.332 [+0.143, +0.522] — the same four differences routing alone gives, since β does not enter that axis — but on Acc@1 by only +0.108 [−0.064, +0.280], with the sign reversing on seed 43, where routing alone gives +0.170 [+0.136, +0.204] and no reversal. The second stage absorbs part of what the routing weight does. We report $w = 0.7$ because it was fixed before that stage existed, by the criterion that all six metrics improve on every seed, which $w = 0.6$ fails on seed 43; for the full method the two are unresolved.

4.7 Limitations

Design choices were explored on ImageNet-R, and CUB-200 also carries development experiments, so neither is an untouched holdout; CIFAR-100 tests transfer, but more seeds do not remove selection bias. The twenty-cell grid, the conditional routing ablation and the weight grid are single-seed, and the isolated routing effect is a few hundredths of a point, small enough that four seeds resolve its sign but not its size. Rows quoted in Table 2 come from another implementation, and on 5-Datasets our result is below the published figure. We claim no consistent retention benefit or non-inferiority, did not split runtime into its phases, and did not measure repair/harm counts.

5 Conclusion

A fixed-feature RP classifier complements HRM-PET at two decisions, the DRM task proposal and final classification, at one extra adapter forward and a ridge fit per task. Acc@1 rises by 0.69–1.50 points over three datasets and four seeds, and in all twenty cells of a single-seed pre-training grid. Run apart over four seeds, routing and the gate contribute on different datasets, and the one dataset the method loses on loses to the gate. The gains do not establish a reduction in forgetting.

Acknowledgment. We thank our supervisor for comments on this manuscript.

References

1. Wang, W., Jia, G., Liu, X., Lin, L., Yang, J.: Hybrid re-matching for continual learning with parameter-efficient tuning. In: NeurIPS (2025)
2. McDonnell, M.D., Gong, D., Parvaneh, A., Abbasnejad, E., van den Hengel, A.: RanPAC: random projections and pre-trained models for continual learning. In: NeurIPS (2023)
3. Wang, L., Xie, J., Zhang, X., Huang, M., Su, H., Zhu, J.: Hierarchical decomposition of prompt-based continual learning: rethinking obscured sub-optimality. In: NeurIPS (2023)
4. Wang, Z., Zhang, Z., Lee, C.Y., Zhang, H., Sun, R., Ren, X., Su, G., Perot, V., Dy, J., Pfister, T.: Learning to prompt for continual learning. In: CVPR (2022)
5. Wang, Z., Zhang, Z., Ebrahimi, S., Sun, R., Zhang, H., Lee, C.Y., Ren, X., Su, G., Perot, V., Dy, J., et al.: DualPrompt: complementary prompting for rehearsal-free continual learning. In: ECCV (2022)
6. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: LoRA: low-rank adaptation of large language models. In: ICLR (2022)
7. Smith, J.S., Karlinsky, L., Gutta, V., Cascante-Bonilla, P., Kim, D., Arbelle, A., Panda, R., Feris, R., Kira, Z.: CODA-Prompt: continual decomposed attention-based prompting for rehearsal-free continual learning. In: CVPR (2023)
8. Zhou, D.W., Cai, Z.W., Ye, H.J., Zhan, D.C., Liu, Z.: Revisiting class-incremental learning with pre-trained models: generalizability and adaptivity are all you need. *Int. J. Comput. Vis.* **133**, 1012–1032 (2025)
9. Masana, M., Liu, X., Twardowski, B., Menta, M., Bagdanov, A.D., van de Weijer, J.: Class-incremental learning: survey and performance evaluation on image classification. *IEEE Trans. Pattern Anal. Mach. Intell.* **45**(5), 5513–5533 (2023)
10. Rebuffi, S.A., Kolesnikov, A., Sperl, G., Lampert, C.H.: iCaRL: incremental classifier and representation learning. In: CVPR (2017)
11. Wu, Y., Chen, Y., Wang, L., Ye, Y., Liu, Z., Guo, Y., Fu, Y.: Large scale incremental learning. In: CVPR (2019)
12. Zhao, B., Xiao, X., Gan, G., Zhang, B., Xia, S.T.: Maintaining discrimination and fairness in class incremental learning. In: CVPR (2020)
13. Hou, S., Pan, X., Loy, C.C., Wang, Z., Lin, D.: Learning a unified classifier incrementally via rebalancing. In: CVPR (2019)
14. Li, L., Wu, Z., Ji, Y.: MoTE: mixture of task-specific experts for pre-trained model based class-incremental learning. *Knowledge-Based Systems* (2025)
15. Zhao, H., Liu, R., Liu, Y.: Dynamic LoRA-experts and prototype-ensemble matching for class-incremental learning. *Applied Sciences* **16**(12), 6153 (2026)
16. Wang, Y., Huang, Z., Hong, X.: S-Prompts learning with pre-trained transformers: an Occam’s razor for domain incremental learning. In: NeurIPS (2022)
17. Gao, Q., Zhao, C., Sun, Y., Xi, T., Zhang, G., Ghanem, B., Zhang, J.: A unified continual learning framework with general parameter-efficient tuning. In: ICCV (2023)
18. Gao, Z., Cen, J., Chang, X.: Consistent prompting for rehearsal-free continual learning. In: CVPR (2024)
19. Liang, Y.S., Li, W.J.: InfLoRA: interference-free low-rank adaptation for continual learning. In: CVPR (2024)
20. Zhuang, H., Weng, Z., Wei, H., Xie, R., Toh, K.A., Lin, Z.: ACIL: analytic class-incremental learning with absolute memorization and privacy protection. In: NeurIPS (2022)

21. Zhuang, H., He, R., Tong, K., Zeng, Z., Chen, C., Lin, Z.: DS-AL: a dual-stream analytic learning for exemplar-free class-incremental learning. In: AAAI (2024)